Logan(10:58):

```
from drafter import *
from dataclasses import dataclass
import math

set_website_title("Roller Coaster G-Force Simulator")
set_site_information(
    author="Logan Gagliardo",
    description="Create different coaster elements and simulate the forces applied to the rider.",
    sources="source",
    planning="Copy of CS1 Final Project 1_ Design.docx",
    links=["https://github.com/logangag/coaster-gforce-simulator"]
)
hide_debug_information()
set_website_framed(False)

# Fixed values used by the simulator
GRAVITY = 9.81
BRAKE_DURATION = 5.0

MAX_VERTICAL_G = 4.5
MIN_VERTICAL_G = -1.0
MAX_LATERAL_G = 1.5
MAX_FORWARD_G = 1.5


@dataclass
class TrackElement:
    name: str
    element_type: str
    parameters: dict[str, float]
    entry_speed: float
    vertical_g: float
    lateral_g: float


@dataclass
class State:
    coaster_name: str
    elements: list[TrackElement]
    current_element_type: str
    current_speed: float
    report: list[str]


# HOME PAGE
@route
def index(state: State) -> Page:
    return Page(state, [
        Header("Roller Coaster G-Force Simulator"),
        "Build your own virtual roller coaster and see how its track elements affect G-forces.",
        "Coaster name:",
        TextBox("coaster_name", state.coaster_name),
        "Choose your first track element:",
        SelectBox("first_element", ["Launch", "90-degree Drop"]),
        Button("Start Building", "begin_coaster")
    ])


# CHOOSE THE FIRST ELEMENT
@route
def begin_coaster(state: State, coaster_name: str,
                  first_element: str) -> Page:
    state.coaster_name = coaster_name
    state.current_element_type = first_element
    state.elements = []
    state.current_speed = 0.0
```

```
    state.report = []

    if first_element == "Launch":
        return launch_page(state)
    else:
        return drop_page(state)


# LAUNCH CONFIGURATION PAGE
@route
def launch_page(state: State) -> Page:
    return Page(state, [
        Header("Configure Your Launch"),
        "Enter the launch acceleration in meters per second squared:",
        TextBox("acceleration", 10.0),
        "Enter how many seconds the launch lasts:",
        TextBox("duration", 3.0),
        Button("Add Launch", "save_launch")
    ])


# SAVE THE LAUNCH
@route
def save_launch(state: State, acceleration: float,
                duration: float) -> Page:

    if acceleration < 0 or duration <= 0:
        return Page(state, [
            Header("Invalid Input"),
            "Acceleration cannot be negative, and duration must be greater than zero.",
            "Enter launch acceleration (m/s²):",
            TextBox("acceleration", acceleration),
            "Enter launch duration (seconds):",
            TextBox("duration", duration),
            Button("Try Again", "save_launch")
        ])

    entry_speed = state.current_speed
    forward_g = acceleration / GRAVITY
    new_speed = entry_speed + acceleration * duration

    launch = TrackElement(
        "Launch", "Launch",
        {
            "acceleration": acceleration,
            "duration": duration,
            "forward_g": forward_g
        },
        entry_speed, 1.0, 0.0
    )

    state.elements.append(launch)
    state.current_speed = new_speed

    return element_selection_page(state)


# DROP CONFIGURATION PAGE
@route
def drop_page(state: State) -> Page:
    return Page(state, [
        Header("Configure Your 90-Degree Drop"),
        "Enter the vertical drop height in meters:",
        TextBox("drop_height", 30.0),
        "Enter the radius of the curve at the bottom in meters:",
        TextBox("curve_radius", 15.0),
        Button("Add Drop", "save_drop")
    ])
```

```
# SAVE THE DROP
@route
def save_drop(state: State, drop_height: float,
              curve_radius: float) -> Page:

    if drop_height < 0 or curve_radius <= 0:
        return Page(state, [
            Header("Invalid Input"),
            "Drop height cannot be negative, and curve radius must be greater than zero.",
            "Enter drop height (meters):",
            TextBox("drop_height", drop_height),
            "Enter curve radius (meters):",
            TextBox("curve_radius", curve_radius),
            Button("Try Again", "save_drop")
        ])

    entry_speed = state.current_speed

    new_speed = math.sqrt(
        entry_speed ** 2 + 2 * GRAVITY * drop_height
    )

    vertical_g = 1 + (
        new_speed ** 2 / (curve_radius * GRAVITY)
    )

    drop = TrackElement(
        "90-Degree Drop", "Drop",
        {
            "drop_height": drop_height,
            "curve_radius": curve_radius
        },
        entry_speed, vertical_g, 0.0
    )

    state.elements.append(drop)
    state.current_speed = new_speed

    return element_selection_page(state)


# CHOOSE ADDITIONAL TRACK ELEMENTS
@route
def element_selection_page(state: State) -> Page:
    return Page(state, [
        Header("Build Your Coaster"),
        "Current coaster: " + state.coaster_name,
        "Current speed: " + str(round(state.current_speed, 2)) + " m/s",
        "Track elements added: " + str(len(state.elements)),
        "Choose your next track element:",
        SelectBox(
            "element_type",
            ["Vertical Loop", "Airtime Hill", "Banked Turn"]
        ),
        Button("Configure Element", "choose_element"),
        Button("Finish Coaster", "finish_track")
    ])


# SELECT AN ELEMENT
@route
def choose_element(state: State, element_type: str) -> Page:
    state.current_element_type = element_type
    return element_config_page(state)


# CONFIGURE AN ADDITIONAL ELEMENT
@route
def element_config_page(state: State) -> Page:
    if state.current_element_type == "Banked Turn":
```

```
        return Page(state, [
            Header("Configure Banked Turn"),
            "Enter the curve radius in meters:",
            TextBox("radius", 20.0),
            "Enter the bank angle in degrees:",
            TextBox("bank_angle", 30.0),
            Button("Add Element", "add_element_action")
        ])
    else:
        return Page(state, [
            Header("Configure " + state.current_element_type),
            "Enter the curve radius in meters:",
            TextBox("radius", 20.0),
            Button("Add Element", "add_element_action")
        ])


# ADD THE ELEMENT AND CALCULATE ITS G-FORCES
@route
def add_element_action(state: State, radius: float,
                       bank_angle: float = 0.0) -> Page:

    print("RADIUS RECEIVED:", radius)

    if radius <= 0:
        return Page(state, [
            Header("Invalid Input"),
            "Radius must be greater than zero.",
            "Enter curve radius (meters):",
            TextBox("radius", 20.0),
            Button("Add Element", "add_element_action")
        ])

    if state.current_element_type == "Banked Turn":
        if bank_angle < 0 or bank_angle > 90:
            return Page(state, [
                Header("Invalid Input"),
                "Bank angle must be between 0 and 90 degrees.",
                "Enter curve radius (meters):",
                TextBox("radius", radius),
                "Enter bank angle (degrees):",
                TextBox("bank_angle", 30.0),
                Button("Add Element", "add_element_action")
            ])

    entry_speed = state.current_speed
    vertical_g = 1.0
    lateral_g = 0.0

    centripetal_g = entry_speed ** 2 / (radius * GRAVITY)

    if state.current_element_type == "Vertical Loop":
        vertical_g = 1 + centripetal_g

    elif state.current_element_type == "Airtime Hill":
        vertical_g = 1 - centripetal_g

    elif state.current_element_type == "Banked Turn":
        lateral_g = centripetal_g * math.sin(
            math.radians(bank_angle)
        )

    element = TrackElement(
        state.current_element_type,
        state.current_element_type,
        {
            "radius": radius,
            "bank_angle": bank_angle
        },
        entry_speed,
```

```
        vertical_g,
        lateral_g
    )

    state.elements.append(element)

    return element_selection_page(state)


# FINISH THE COASTER AND AUTOMATICALLY ADD A BRAKE RUN
@route
def finish_track(state: State) -> Page:

    # The brake run is automatically added as the final element.
    entry_speed = state.current_speed
    deceleration = entry_speed / BRAKE_DURATION
    forward_g = -deceleration / GRAVITY

    brake_run = TrackElement(
        "Required Brake Run",
        "Brake Run",
        {
            "deceleration": deceleration,
            "duration": BRAKE_DURATION,
            "forward_g": forward_g
        },
        entry_speed,
        1.0,
        0.0
    )

    state.elements.append(brake_run)
    state.current_speed = 0.0

    return report_page(state)


# FINAL REPORT PAGE
@route
def report_page(state: State) -> Page:

    state.report = []

    state.report.append("Coaster: " + state.coaster_name)
    state.report.append(
        "Total track elements: " + str(len(state.elements))
    )
    state.report.append("")

    overall_safe = True

    # Loop through every track element and evaluate its G-forces.
    for element in state.elements:

        state.report.append(element.name)

        state.report.append(
            "Entry speed: " + str(round(element.entry_speed, 2))
            + " m/s"
        )

        state.report.append(
            "Vertical G-force: " + str(round(element.vertical_g, 2))
        )

        state.report.append(
            "Lateral G-force: " + str(round(element.lateral_g, 2))
        )

        # Check the vertical G-force limits.
```

```
        if (element.vertical_g > MAX_VERTICAL_G or
                element.vertical_g < MIN_VERTICAL_G):
            state.report.append("Vertical G-force: EXCEEDS PROGRAM LIMIT")
            overall_safe = False
        else:
            state.report.append("Vertical G-force: Within program limits")

        # Check the lateral G-force limit.
        if abs(element.lateral_g) > MAX_LATERAL_G:
            state.report.append("Lateral G-force: EXCEEDS PROGRAM LIMIT")
            overall_safe = False
        else:
            state.report.append("Lateral G-force: Within program limits")

        # Check the forward/backward G-force, if the element has one.
        forward_g = element.parameters.get("forward_g", 0.0)

        if abs(forward_g) > MAX_FORWARD_G:
            state.report.append("Forward/backward G-force: EXCEEDS PROGRAM LIMIT")
            overall_safe = False
        else:
            state.report.append("Forward/backward G-force: Within program limits")

        state.report.append("")

    if overall_safe:
        state.report.append("Overall Result: Within this program's thresholds.")
    else:
        state.report.append("Overall Result: Some elements exceed this program's thresholds.")

    return Page(state, [
        Header("Final Coaster Report"),
        Header(state.coaster_name, 2),
        "The following report evaluates each track element:",
        BulletedList(state.report),
        Button("Start a New Coaster", "restart_action")
    ])


# RESTART THE SIMULATOR
@route
def restart_action(state: State) -> Page:
    state.coaster_name = ""
    state.elements = []
    state.current_element_type = ""
    state.current_speed = 0.0
    state.report = []

    return index(state)


start_server(State(
    "",
    [],
    "",
    0.0,
    []
))
```

rate my code and see if it does everything on the critera for my final project.

Site Functionality Requirements

Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

- At least 7 routes.

- At least 5 Pages, or 10 FragmentsLinks to an external site., or a combination thereof.
- The implementation of the *State* dataclass must have at least 4 fields.
    - At least one field must be a list of class instances or a dictionary of class instances.
        - This means that there must be a second dataclass.
    - At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
- At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
- At least 3 meaningful if statements.
- At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
- The site should have a legitimate purpose or functionality.
- No global variables (global constantsLinks to an external site. are fine).
- You must use the Drafter library.
- Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

More creative and impressive designs are given flexibility on the above requirements - impress us! However, impressive websites tend to do more than the above requirements. Further, **if you miss more than 2 requirements, you will get no credit for that submission**. For example, missing a second dataclass means you would be missing not only that requirement, but also the list or dictionary of such dataclass instances, and the loop iterating over that list. This would mean that the submission would get a zero.

Code Requirements

Make sure your program and website runs; **broken programs do not earn points**. This means that if your program contains a syntax error or runs and doesn't deploy a website, you get zero points for this assignment, and are subject to any other penalties listed in the syllabus.

Once you are finished with your project code, check over the following:

- Chosen good variable names (check last two sections of Chapter 1B1Links to an external site.)
- Unit tested your functions where possible, being valid and thoroughLinks to an external site.: each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).
    - If a function can't be unit tested, you must note as much in a docstring or comment.
- Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)
- Included types in your function headers
- Included good docstrings Links to an external site.as the first line of your function's body
- Decomposed complicated functions with helper functions

Your TA will be grading your code's quality based on the ideas above.

~~ChatGPT(11:00):~~

I'd rate this about 7/10 as submitted. You appear to meet the core site functionality requirements. The biggest grading risks are missing tests, missing docstrings, and a calculation bug.

I checked Python syntax, started it with Drafter 2.2.1, confirmed the homepage returned HTTP 200, and directly exercised all 13 routes. Those checks passed for the inputs tested; I haven't verified every interaction in a browser.

Site functionality checklist

Requirement	Result	Evidence
At least 7 routes	✅	13 decorated routes
At least 5 pages or equivalent	✅	Six page-producing routes: home, launch, drop, element selection, element configuration, report
State has at least 4 fields	✅	Five fields
Second dataclass and collection of instances	✅	`TrackElement` and `elements: list[TrackElement]`
At least 4 state fields meaningfully modified	✅	All five change during the application flow

At least 3 input fields	✅	Name, element selections, acceleration, duration, height, radius, bank angle
At least 3 meaningful `if` statements	✅	Input validation, element calculations, threshold checks
Meaningful loop over stored class instances	✅	`report_page` loops over `state.elements` and evaluates their attributes
Legitimate purpose	✅	Builds a coaster and evaluates calculated forces
No global variables	✅	Your module-level numeric values are constants
Uses Drafter	✅	Uses routes, pages, inputs, and server startup
Advanced techniques documented	⚠️	Depends on what your course covered; explain unfamiliar techniques and physics formulas

I don't see evidence that you're missing more than two functionality requirements. However, code quality is assessed separately.

Code quality checklist

Requirement	Result
Good variable names	✅ Clear and descriptive
Unit tests for each route where possible	❌ None in the uploaded code
Lines no longer than 120 characters	✅ Longest line: 98 characters
Function parameter and return types	✅ All routes have annotations
Docstrings inside functions	❌ None of the 13 routes has one
Helpers for complicated functions	⚠️ Split calculation and reporting work into helpers

Your comments above functions **do not count as docstrings**. For example:

```
None
@route
def restart_action(state: State) -> Page:
    """Clear the current coaster and return to the home page."""
```

Fix these before submitting

1. **Add tests for all 13 routes.** These routes are deterministic, so they appear testable. Include successful actions, invalid inputs, state changes, and threshold boundaries. My temporary review checks don't count as tests included in your submission.
2. **Fix the banked-turn formula around lines 264–267.** Currently:

```
None
```

3.
```
    lateral_g = centripetal_g * math.sin(math.radians(bank_angle))
```

4.
    At a bank angle of zero, this always produces zero lateral G—even for a fast, sharp turn. For a simplified level turn, with forces measured relative to the rider:

None

```
angle = math.radians(bank_angle)
lateral_g = centripetal_g * math.cos(angle) - math.sin(angle)

    5.  vertical_g = math.cos(angle) + centripetal_g * math.sin(angle)
```

6. Document the model and its assumptions.
7. **Preserve the bank angle after an invalid radius.** The error page at lines 231–238 omits its input. On resubmission, `bank_angle` defaults to `0.0`, silently changing the user's turn.
8. **Add docstrings and extract helpers.** Good candidates are a force-calculation helper and a helper that evaluates one track element for the report.
9. **Explain the simulation's simplifications.** Your vertical loop evaluates the bottom force; it doesn't evaluate the top or whether the coaster has enough speed to complete the loop. Speeds also remain unchanged through additional elements. Label those assumptions clearly.

Remove the debugging `print` at line 229 as a final cleanup. **The project structure meets the assignment well; tests, documentation, and the banked-turn fix should be your priorities.**

Logan(11:01):

rewrite the code with all the fixs, including the unit tests and docstrings. For the unit tests, use from bakery import assert_equal

ChatGPT(11:16):

Here's a complete replacement. It includes the fixes, documented formulas, docstrings, and `from bakery import assert_equal`.

Verified: all 135 Bakery assertions pass, all 13 routes have tests, and the server starts with a working homepage.

"""Build a coaster and compare idealized rider forces with program limits.

Physics model:

* Ignore friction and air resistance.

* A drop converts gravitational potential energy into kinetic energy.

* A circular loop is evaluated at its bottom and top. It returns to its

  starting height, so its exit speed equals its entry speed.

* An airtime hill evaluates the crest only. Approach and descent return to

  the same height, so its exit speed also equals its entry speed.

* A banked turn is level. Rotate gravity and centripetal acceleration into

  the rider's vertical and lateral axes.

* Launch and brake acceleration are constant over their duration.

* These are educational estimates, not real ride safety assessments.

Python techniques used:

```
* list[TrackElement] stores dataclass instances.

* dict[str, float] stores named numeric parameters.

* math.isfinite rejects infinity and NaN.

* math.radians converts degrees for math.sin and math.cos.

* Dictionary.get supplies a default for an optional parameter.

* A default bank_angle argument lets different elements share one route.

* String.strip removes surrounding spaces.

* List.extend adds several report lines; append adds one item.

* in/not in check supported choices and optional dictionary keys.

* Chained comparisons check inclusive ranges.

* Index -1 accesses the final element after checking the list is not empty.

* Tests create local State objects without changing the live website.

"""


from drafter import *

from bakery import assert_equal

from dataclasses import dataclass

import math




set_website_title("Roller Coaster G-Force Simulator")

set_site_information(

    author="Logan Gagliardo",

    description="Create coaster elements and simulate forces on the rider.",

    sources="Idealized formulas from Newton's laws and conservation of energy.",

    planning="Copy of CS1 Final Project 1_ Design.docx",

    links=["https://github.com/logangag/coaster-gforce-simulator"]

)

hide_debug_information()

set_website_framed(False)
```

```
# Global constants are allowed by the project requirements.
GRAVITY = 9.81
BRAKE_DURATION = 5.0

MAX_VERTICAL_G = 4.5
MIN_VERTICAL_G = -1.0
MAX_LATERAL_G = 1.5
MAX_FORWARD_G = 1.5



@dataclass
class TrackElement:
    """Store one element and its calculated rider forces in units of g."""

    name: str
    element_type: str
    parameters: dict[str, float]
    entry_speed: float
    vertical_g: float
    lateral_g: float



@dataclass
class State:
    """Store the coaster, selected element, current speed, and report."""

    coaster_name: str
    elements: list[TrackElement]
    current_element_type: str
    current_speed: float
```

```
    report: list[str]



# HELPER FUNCTIONS


def initial_state() -> State:
    """Create an empty coaster with fresh lists."""
    return State("", [], "", 0.0, [])



def positive_number(value: float) -> bool:
    """Return whether a number is finite and greater than zero."""
    return math.isfinite(value) and value > 0



def invalid_selection(state: State) -> Page:
    """Explain an unsupported selection and offer a return button."""
    return Page(state, [
        Header("Invalid Selection"),
        "Choose an element from the available options.",
        Button("Return to Builder", "element_selection_page")
    ])



def element_error(state: State, message: str, radius: float,
                  bank_angle: float) -> Page:
    """Show an error while preserving the submitted radius and bank angle."""
    content = [
        Header("Invalid Input"),
        message,
        "Enter curve radius (meters):",
```

```
        TextBox("radius", radius)
    ]

    if state.current_element_type == "Banked Turn":
        content.append("Enter bank angle (degrees):")
        content.append(TextBox("bank_angle", bank_angle))

    content.append(Button("Try Again", "add_element_action"))

    return Page(state, content)



def calculate_element(element_type: str, speed: float, radius: float,
                      bank_angle: float) -> TrackElement:
    """Calculate forces for an already validated loop, hill, or level turn.

    Centripetal acceleration is speed squared divided by radius.
    Dividing by GRAVITY expresses it in units of g.

    A circular loop's top is 2 * radius above its bottom. Conservation
    of energy gives top_speed_squared = speed squared - 4 * g * radius.
    At the top, gravity points toward the center, so subtract 1 g when
    calculating the seat's force on the rider.
    """
    centripetal_g = speed ** 2 / (radius * GRAVITY)
    vertical_g = 1.0
    lateral_g = 0.0

    parameters = {
        "radius": radius,
        "bank_angle": bank_angle
```

```
}

if element_type == "Vertical Loop":
    vertical_g = 1 + centripetal_g

    top_speed_squared = speed ** 2 - 4 * GRAVITY * radius

    parameters["top_speed"] = math.sqrt(top_speed_squared)
    parameters["top_vertical_g"] = (
        top_speed_squared / (radius * GRAVITY) - 1
    )

elif element_type == "Airtime Hill":
    vertical_g = 1 - centripetal_g

elif element_type == "Banked Turn":
    angle = math.radians(bank_angle)

    vertical_g = (
        math.cos(angle) + centripetal_g * math.sin(angle)
    )
    lateral_g = (
        centripetal_g * math.cos(angle) - math.sin(angle)
    )

return TrackElement(
    element_type,
    element_type,
    parameters,
    speed,
    vertical_g,
```

```
        lateral_g
    )


def within_limits(element: TrackElement) -> bool:
    """Check vertical, lateral, longitudinal, and optional loop-top forces."""
    if not MIN_VERTICAL_G <= element.vertical_g <= MAX_VERTICAL_G:
        return False

    if abs(element.lateral_g) > MAX_LATERAL_G:
        return False

    if abs(element.parameters.get("forward_g", 0.0)) > MAX_FORWARD_G:
        return False

    if "top_vertical_g" in element.parameters:
        top_g = element.parameters["top_vertical_g"]

        if not MIN_VERTICAL_G <= top_g <= MAX_VERTICAL_G:
            return False

    return True


def describe_element(element: TrackElement) -> list[str]:
    """Build report lines showing an element's forces and threshold result."""
    lines = [
        element.name,
        "Entry speed: " + str(round(element.entry_speed, 2)) + " m/s",
        "Vertical G-force (bottom/crest/level): "
        + str(round(element.vertical_g, 2)),
```

```
        "Lateral G-force: " + str(round(element.lateral_g, 2))
    ]

    if "forward_g" in element.parameters:
        lines.append(
            "Forward/backward G-force: "
            + str(round(element.parameters["forward_g"], 2))
        )

    if "top_vertical_g" in element.parameters:
        lines.append(
            "Loop top speed: "
            + str(round(element.parameters["top_speed"], 2))
            + " m/s"
        )
        lines.append(
            "Loop top vertical G-force: "
            + str(round(element.parameters["top_vertical_g"], 2))
        )

    if within_limits(element):
        lines.append("Element result: Within program limits")
    else:
        lines.append("Element result: EXCEEDS PROGRAM LIMIT")

    return lines


# HOME PAGE

@route
```

```
def index(state: State) -> Page:
    """Display the coaster name input and starting-element selection."""
    return Page(state, [
        Header("Roller Coaster G-Force Simulator"),
        "Build your own virtual roller coaster and see how its track elements affect G-forces.",
        "Educational model: no friction or drag. Results are not a ride safety assessment.",
        "Loops and hills return to their entry height; hill force is evaluated at the crest.",
        "Coaster name:",
        TextBox("coaster_name", state.coaster_name),
        "Choose your first track element:",
        SelectBox("first_element", ["Launch", "90-degree Drop"]),
        Button("Start Building", "begin_coaster")
    ])



@route
def begin_coaster(state: State, coaster_name: str,
                  first_element: str) -> Page:
    """Reset the coaster and open the selected starting-element form."""
    if first_element != "Launch" and first_element != "90-degree Drop":
        return invalid_selection(state)

    state.coaster_name = coaster_name.strip()

    if state.coaster_name == "":
        state.coaster_name = "Untitled Coaster"

    state.current_element_type = first_element
    state.elements = []
    state.current_speed = 0.0
    state.report = []
```

```
    if first_element == "Launch":
        return launch_page(state)
    else:
        return drop_page(state)



# LAUNCH


@route
def launch_page(state: State) -> Page:
    """Display inputs for constant launch acceleration and duration."""
    return Page(state, [
        Header("Configure Your Launch"),
        "Enter the launch acceleration in meters per second squared:",
        TextBox("acceleration", 10.0),
        "Enter how many seconds the launch lasts:",
        TextBox("duration", 3.0),
        Button("Add Launch", "save_launch")
    ])



@route
def save_launch(state: State, acceleration: float,
                duration: float) -> Page:
    """Validate launch inputs, save the launch, and update the speed."""
    if not positive_number(acceleration) or not positive_number(duration):
        return Page(state, [
            Header("Invalid Input"),
            "Acceleration and duration must be finite numbers greater than zero.",
            "Enter launch acceleration (m/s²):",
```

```
            TextBox("acceleration", acceleration),
            "Enter launch duration (seconds):",
            TextBox("duration", duration),
            Button("Try Again", "save_launch")
        ])

    entry_speed = state.current_speed
    forward_g = acceleration / GRAVITY
    new_speed = entry_speed + acceleration * duration

    launch = TrackElement(
        "Launch",
        "Launch",
        {
            "acceleration": acceleration,
            "duration": duration,
            "forward_g": forward_g
        },
        entry_speed,
        1.0,
        0.0
    )

    state.elements.append(launch)
    state.current_speed = new_speed

    return element_selection_page(state)


# DROP
```

```
@route
def drop_page(state: State) -> Page:
    """Display inputs for drop height and bottom curve radius."""
    return Page(state, [
        Header("Configure Your 90-Degree Drop"),
        "Enter the vertical drop height in meters:",
        TextBox("drop_height", 30.0),
        "Enter the radius of the curve at the bottom in meters:",
        TextBox("curve_radius", 15.0),
        Button("Add Drop", "save_drop")
    ])



@route
def save_drop(state: State, drop_height: float,
              curve_radius: float) -> Page:
    """Validate drop inputs and calculate bottom speed and rider force."""
    if not positive_number(drop_height) or not positive_number(curve_radius):
        return Page(state, [
            Header("Invalid Input"),
            "Drop height and curve radius must be finite numbers greater than zero.",
            "Enter drop height (meters):",
            TextBox("drop_height", drop_height),
            "Enter curve radius (meters):",
            TextBox("curve_radius", curve_radius),
            Button("Try Again", "save_drop")
        ])

    entry_speed = state.current_speed

    # Conservation of energy: final speed squared = initial speed squared
```

```
    # plus twice gravity times the vertical drop height.
    new_speed = math.sqrt(
        entry_speed ** 2 + 2 * GRAVITY * drop_height
    )

    # At the bottom, the seat supports gravity and supplies the additional
    # upward centripetal acceleration.
    vertical_g = 1 + new_speed ** 2 / (curve_radius * GRAVITY)

    drop = TrackElement(
        "90-Degree Drop",
        "Drop",
        {
            "drop_height": drop_height,
            "curve_radius": curve_radius
        },
        entry_speed,
        vertical_g,
        0.0
    )

    state.elements.append(drop)
    state.current_speed = new_speed

    return element_selection_page(state)


# ADDITIONAL ELEMENTS

@route
def element_selection_page(state: State) -> Page:
```

```
    """Show the current coaster and choices for its next element."""
    return Page(state, [
        Header("Build Your Coaster"),
        "Current coaster: " + state.coaster_name,
        "Current speed: " + str(round(state.current_speed, 2)) + " m/s",
        "Track elements added: " + str(len(state.elements)),
        "Choose your next track element:",
        SelectBox(
            "element_type",
            ["Vertical Loop", "Airtime Hill", "Banked Turn"]
        ),
        Button("Configure Element", "choose_element"),
        Button("Finish Coaster", "finish_track")
    ])



@route
def choose_element(state: State, element_type: str) -> Page:
    """Save a supported element selection and display its input form."""
    if element_type not in ["Vertical Loop", "Airtime Hill", "Banked Turn"]:
        return invalid_selection(state)

    state.current_element_type = element_type

    return element_config_page(state)



@route
def element_config_page(state: State) -> Page:
    """Display radius and, for a banked turn, bank-angle inputs."""
    if state.current_element_type == "Banked Turn":
```

```
        return Page(state, [
            Header("Configure Banked Turn"),
            "Enter the curve radius in meters:",
            TextBox("radius", 20.0),
            "Enter the bank angle in degrees:",
            TextBox("bank_angle", 30.0),
            Button("Add Element", "add_element_action")
        ])
    else:
        return Page(state, [
            Header("Configure " + state.current_element_type),
            "Enter the curve radius in meters:",
            TextBox("radius", 20.0),
            Button("Add Element", "add_element_action")
        ])



@route
def add_element_action(state: State, radius: float,
                       bank_angle: float = 0.0) -> Page:
    """Validate geometry and loop speed, then append the calculated element."""
    if not positive_number(radius):
        return element_error(
            state,
            "Radius must be a finite number greater than zero.",
            radius,
            bank_angle
        )

    if state.current_element_type not in [
        "Vertical Loop", "Airtime Hill", "Banked Turn"
```

```
]:
    return invalid_selection(state)


if state.current_element_type == "Banked Turn":
    if not math.isfinite(bank_angle) or bank_angle < 0 or bank_angle > 90:
        return element_error(
            state,
            "Bank angle must be a finite number from 0 to 90 degrees.",
            radius,
            bank_angle
        )


if state.current_element_type == "Vertical Loop":
    # An unpowered circular loop needs v squared >= 5*g*r to keep
    # nonnegative seat force at the top under this idealized model.
    if state.current_speed ** 2 < 5 * GRAVITY * radius:
        return element_error(
            state,
            "Too slow to complete this circular loop. Use a smaller radius.",
            radius,
            bank_angle
        )


element = calculate_element(
    state.current_element_type,
    state.current_speed,
    radius,
    bank_angle
)


state.elements.append(element)
```

```
    return element_selection_page(state)



# FINISH AND REPORT


@route
def finish_track(state: State) -> Page:
    """Add one final brake run, stop the coaster, and show its report."""
    if not state.elements:
        return index(state)


    # Repeated submissions should not add duplicate brake runs.
    if state.elements[-1].element_type == "Brake Run":
        return report_page(state)


    entry_speed = state.current_speed
    deceleration = entry_speed / BRAKE_DURATION
    forward_g = -deceleration / GRAVITY


    brake_run = TrackElement(
        "Required Brake Run",
        "Brake Run",
        {
            "deceleration": deceleration,
            "duration": BRAKE_DURATION,
            "forward_g": forward_g
        },
        entry_speed,
        1.0,
        0.0
```

```
    )

    state.elements.append(brake_run)
    state.current_speed = 0.0

    return report_page(state)


@route
def report_page(state: State) -> Page:
    """Rebuild the report by evaluating each stored track element."""
    state.report = [
        "Coaster: " + state.coaster_name,
        "Total track elements: " + str(len(state.elements))
    ]

    overall_safe = True

    # This loop reads attributes of the TrackElement instances in State.
    for element in state.elements:
        state.report.extend(describe_element(element))

        if not within_limits(element):
            overall_safe = False

    if overall_safe:
        state.report.append(
            "Overall Result: Within this program's thresholds."
        )
    else:
        state.report.append(
```

```
            "Overall Result: Some elements exceed this program's thresholds."
        )

    return Page(state, [
        Header("Final Coaster Report"),
        Header(state.coaster_name, 2),
        "These are idealized estimates, not an assessment of real ride safety.",
        "The following report evaluates each track element:",
        BulletedList(state.report),
        Button("Start a New Coaster", "restart_action")
    ])


@route
def restart_action(state: State) -> Page:
    """Clear every state field and return to the home page."""
    state.coaster_name = ""
    state.elements = []
    state.current_element_type = ""
    state.current_speed = 0.0
    state.report = []

    return index(state)


# UNIT TESTS
# Every route is deterministic and can be unit tested.
# Tests run before the server starts and use separate local State objects.
# Round calculated floats to avoid insignificant representation differences.
# float("inf") and float("nan") create special values for validation tests.
```

```
def test_index() -> None:
    """Check that the home page contains its name input and start action."""
    state = initial_state()
    page = index(state)

    assert_equal(page.state, state)
    assert_equal(
        page.content[0],
        Header("Roller Coaster G-Force Simulator")
    )
    assert_equal(page.content[5], TextBox("coaster_name", ""))
    assert_equal(
        page.content[-1],
        Button("Start Building", "begin_coaster")
    )



def test_begin_coaster() -> None:
    """Check starting choices, reset behavior, and blank-name handling."""
    state = State("Old", [], "Banked Turn", 50.0, ["Old report"])
    state.elements.append(
        TrackElement("Old", "Old", {}, 0.0, 1.0, 0.0)
    )

    page = begin_coaster(state, "  Comet  ", "Launch")

    assert_equal(state, State("Comet", [], "Launch", 0.0, []))
    assert_equal(page.content[0], Header("Configure Your Launch"))

    page = begin_coaster(state, "    ", "90-degree Drop")
```

```
    assert_equal(state.coaster_name, "Untitled Coaster")
    assert_equal(
        page.content[0],
        Header("Configure Your 90-Degree Drop")
    )

    page = begin_coaster(state, "Ignored", "Unsupported")

    assert_equal(page.content[0], Header("Invalid Selection"))
    assert_equal(state.coaster_name, "Untitled Coaster")



def test_launch_page() -> None:
    """Check the launch form's inputs and destination route."""
    state = initial_state()

    assert_equal(launch_page(state), Page(state, [
        Header("Configure Your Launch"),
        "Enter the launch acceleration in meters per second squared:",
        TextBox("acceleration", 10.0),
        "Enter how many seconds the launch lasts:",
        TextBox("duration", 3.0),
        Button("Add Launch", "save_launch")
    ]))



def test_save_launch() -> None:
    """Check launch speed, stored parameters, and invalid input rejection."""
    state = initial_state()
    page = save_launch(state, 10.0, 3.0)
```

```
assert_equal(state.current_speed, 30.0)
assert_equal(state.elements, [TrackElement(
    "Launch",
    "Launch",
    {
        "acceleration": 10.0,
        "duration": 3.0,
        "forward_g": 10.0 / GRAVITY
    },
    0.0,
    1.0,
    0.0
)])
assert_equal(page.content[0], Header("Build Your Coaster"))

for acceleration in [-1.0, 0.0, float("inf"), float("nan")]:
    page = save_launch(state, acceleration, 3.0)

    assert_equal(page.content[0], Header("Invalid Input"))
    assert_equal(len(state.elements), 1)
    assert_equal(state.current_speed, 30.0)

page = save_launch(state, 10.0, 0.0)

assert_equal(page.content[0], Header("Invalid Input"))
assert_equal(len(state.elements), 1)

save_launch(state, 2.0, 5.0)

assert_equal(state.elements[-1].entry_speed, 30.0)
assert_equal(state.current_speed, 40.0)
```

```
def test_drop_page() -> None:
    """Check the drop form's inputs and destination route."""
    state = initial_state()

    assert_equal(drop_page(state), Page(state, [
        Header("Configure Your 90-Degree Drop"),
        "Enter the vertical drop height in meters:",
        TextBox("drop_height", 30.0),
        "Enter the radius of the curve at the bottom in meters:",
        TextBox("curve_radius", 15.0),
        Button("Add Drop", "save_drop")
    ]))



def test_save_drop() -> None:
    """Check energy-based speed, bottom force, and invalid geometry."""
    state = initial_state()
    page = save_drop(state, 30.0, 15.0)

    assert_equal(round(state.current_speed, 4), 24.2611)
    assert_equal(state.elements[0].vertical_g, 5.0)
    assert_equal(state.elements[0].entry_speed, 0.0)
    assert_equal(page.content[0], Header("Build Your Coaster"))

    for height in [-1.0, 0.0, float("inf"), float("nan")]:
        page = save_drop(state, height, 15.0)

        assert_equal(page.content[0], Header("Invalid Input"))
        assert_equal(len(state.elements), 1)
```

```
    for radius in [-1.0, 0.0, float("inf"), float("nan")]:
        page = save_drop(state, 30.0, radius)

        assert_equal(page.content[0], Header("Invalid Input"))
        assert_equal(len(state.elements), 1)

    state.current_speed = 10.0
    save_drop(state, 10.0, 20.0)

    assert_equal(round(state.current_speed, 4), 17.2105)



def test_element_selection_page() -> None:
    """Check that builder details reflect the current state."""
    state = State("Comet", [], "Launch", 30.0, [])
    page = element_selection_page(state)

    assert_equal(page.content[1], "Current coaster: Comet")
    assert_equal(page.content[2], "Current speed: 30.0 m/s")
    assert_equal(page.content[3], "Track elements added: 0")
    assert_equal(
        page.content[-1],
        Button("Finish Coaster", "finish_track")
    )



def test_choose_element() -> None:
    """Check that valid selections change state and invalid ones do not."""
    state = initial_state()
```

```
    for kind in ["Vertical Loop", "Airtime Hill", "Banked Turn"]:
        page = choose_element(state, kind)

        assert_equal(state.current_element_type, kind)
        assert_equal(page.content[0], Header("Configure " + kind))

    page = choose_element(state, "Unsupported")

    assert_equal(state.current_element_type, "Banked Turn")
    assert_equal(page.content[0], Header("Invalid Selection"))



def test_element_config_page() -> None:
    """Check that only a banked turn displays an angle field."""
    state = initial_state()
    state.current_element_type = "Banked Turn"

    page = element_config_page(state)

    assert_equal(page.content[4], TextBox("bank_angle", 30.0))
    assert_equal(len(page.content), 6)

    state.current_element_type = "Vertical Loop"
    page = element_config_page(state)

    assert_equal(page.content[2], TextBox("radius", 20.0))
    assert_equal(len(page.content), 4)



def test_add_element_action() -> None:
    """Check force models, loop feasibility, and preserved error inputs."""
```

```
state = State("Comet", [], "Banked Turn", 30.0, [])
page = add_element_action(state, 20.0, 0.0)

assert_equal(page.content[0], Header("Build Your Coaster"))
assert_equal(round(state.elements[-1].lateral_g, 4), 4.5872)
assert_equal(state.elements[-1].vertical_g, 1.0)

add_element_action(state, 20.0, 30.0)

assert_equal(round(state.elements[-1].lateral_g, 4), 3.4726)
assert_equal(round(state.elements[-1].vertical_g, 4), 3.1596)

page = add_element_action(state, 0.0, 45.0)

assert_equal(page.content[5], TextBox("bank_angle", 45.0))
assert_equal(len(state.elements), 2)

for angle in [-1.0, 91.0, float("inf"), float("nan")]:
    page = add_element_action(state, 20.0, angle)

    assert_equal(page.content[0], Header("Invalid Input"))
    assert_equal(len(state.elements), 2)

for radius in [-1.0, float("inf"), float("nan")]:
    page = add_element_action(state, radius, 45.0)

    assert_equal(page.content[0], Header("Invalid Input"))
    assert_equal(len(state.elements), 2)

state.current_element_type = "Airtime Hill"
add_element_action(state, 20.0)
```

```
    assert_equal(round(state.elements[-1].vertical_g, 4), -3.5872)

    state.current_element_type = "Vertical Loop"
    page = add_element_action(state, 20.0)

    assert_equal(page.content[0], Header("Invalid Input"))
    assert_equal(len(state.elements), 3)

    add_element_action(state, 10.0)

    assert_equal(round(state.elements[-1].vertical_g, 4), 10.1743)
    assert_equal(
        round(state.elements[-1].parameters["top_vertical_g"], 4),
        4.1743
    )
    assert_equal(
        round(state.elements[-1].parameters["top_speed"], 4),
        22.5300
    )
    assert_equal(state.current_speed, 30.0)

    state.current_element_type = "Unsupported"
    page = add_element_action(state, 10.0)

    assert_equal(page.content[0], Header("Invalid Selection"))
    assert_equal(len(state.elements), 4)



def test_finish_track() -> None:
    """Check braking, stopping, and protection from duplicate brake runs."""
```

```
    state = initial_state()
    page = finish_track(state)

    assert_equal(
        page.content[0],
        Header("Roller Coaster G-Force Simulator")
    )
    assert_equal(state.elements, [])

    save_launch(state, 10.0, 3.0)
    page = finish_track(state)

    assert_equal(page.content[0], Header("Final Coaster Report"))
    assert_equal(len(state.elements), 2)
    assert_equal(state.current_speed, 0.0)
    assert_equal(state.elements[-1].element_type, "Brake Run")
    assert_equal(state.elements[-1].entry_speed, 30.0)
    assert_equal(state.elements[-1].parameters["deceleration"], 6.0)
    assert_equal(
        round(state.elements[-1].parameters["forward_g"], 4),
        -0.6116
    )

    finish_track(state)

    assert_equal(len(state.elements), 2)



def test_report_page() -> None:
    """Check passing and failing reports and rebuilding without duplicates."""
    state = State(
```

```
    "Comet",
    [TrackElement("Gentle", "Banked Turn", {}, 10.0, 1.0, 0.5)],
    "Banked Turn",
    10.0,
    []
)
page = report_page(state)

assert_equal(page.content[0], Header("Final Coaster Report"))
assert_equal(state.report[0], "Coaster: Comet")
assert_equal(
    state.report[-1],
    "Overall Result: Within this program's thresholds."
)
assert_equal("Gentle" in state.report, True)
assert_equal(len(state.report), 8)

report_page(state)

assert_equal(len(state.report), 8)

state.elements.append(
    TrackElement("Sharp", "Airtime Hill", {}, 30.0, -2.0, 0.0)
)
report_page(state)

assert_equal("Sharp" in state.report, True)
assert_equal(
    "Element result: EXCEEDS PROGRAM LIMIT" in state.report,
    True
)
```

```
    assert_equal(
        state.report[-1],
        "Overall Result: Some elements exceed this program's thresholds."
    )



def test_restart_action() -> None:
    """Check that restart clears every field and returns the home page."""
    state = State(
        "Comet",
        [TrackElement("Old", "Launch", {}, 0.0, 1.0, 0.0)],
        "Launch",
        30.0,
        ["Old report"]
    )
    page = restart_action(state)

    assert_equal(state, initial_state())
    assert_equal(
        page.content[0],
        Header("Roller Coaster G-Force Simulator")
    )



def test_helpers() -> None:
    """Check inclusive limits, loop-top limits, and balanced banking."""
    assert_equal(positive_number(0.0), False)
    assert_equal(positive_number(2.0), True)
    assert_equal(positive_number(float("nan")), False)
    assert_equal(positive_number(float("inf")), False)
```

```
element = TrackElement(
    "Test", "Test", {"forward_g": 1.5}, 10.0, 4.5, 1.5
)
assert_equal(within_limits(element), True)

element.vertical_g = -1.0
assert_equal(within_limits(element), True)

element.vertical_g = -1.01
assert_equal(within_limits(element), False)

element.vertical_g = 4.51
assert_equal(within_limits(element), False)

element.vertical_g = 1.0
element.lateral_g = -1.51
assert_equal(within_limits(element), False)

element.lateral_g = -1.5
element.parameters["forward_g"] = -1.5
assert_equal(within_limits(element), True)

element.parameters["forward_g"] = -1.51
assert_equal(within_limits(element), False)

element.parameters = {
    "top_vertical_g": 4.51,
    "top_speed": 20.0
}
assert_equal(within_limits(element), False)
```

```
    element.parameters["top_vertical_g"] = -1.01
    assert_equal(within_limits(element), False)

    # Centripetal acceleration is exactly 1 g here. Banking 45 degrees
    # removes lateral force and produces sqrt(2) g in the vertical axis.
    balanced = calculate_element(
        "Banked Turn",
        math.sqrt(20 * GRAVITY),
        20.0,
        45.0
    )

    assert_equal(round(balanced.lateral_g, 4), 0.0)
    assert_equal(round(balanced.vertical_g, 4), 1.4142)
    assert_equal(
        "Element result: EXCEEDS PROGRAM LIMIT"
        in describe_element(element),
        True
    )



def run_tests() -> None:
    """Run Bakery tests for all thirteen routes and the helper functions."""
    test_index()
    test_begin_coaster()
    test_launch_page()
    test_save_launch()
    test_drop_page()
    test_save_drop()
    test_element_selection_page()
    test_choose_element()
```

```
    test_element_config_page()

    test_add_element_action()

    test_finish_track()

    test_report_page()

    test_restart_action()

    test_helpers()




run_tests()

start_server(initial_state())
```

Logan(11:17):

this look awesome, can you make the report screen look better than just a bulleted list, hwo would you do this?

ChatGPT(11:18):

I'd use **an overall result at the top, followed by a separate section for each element**. Each section would show its entry speed, a table of forces, and its result.

Replace only your existing `report_page` with this version. It preserves `state.report`, so your existing report tests still work.

```
@route

def report_page(state: State) -> Page:

    """Display a summary and a force table for each track element.



    Keep the text report in State for testing and future export features.

    Table uses rows of cell values and a separate header row.

    HorizontalRule visually separates the element sections.

    """

    state.report = [

        "Coaster: " + state.coaster_name,

        "Total track elements: " + str(len(state.elements))

    ]



    overall_safe = True

    elements_over_limit = 0
```

```
for element in state.elements:
    state.report.extend(describe_element(element))

    if not within_limits(element):
        overall_safe = False
        elements_over_limit += 1

if overall_safe:
    overall_result = (
        "Overall Result: Within this program's thresholds."
    )
else:
    overall_result = (
        "Overall Result: Some elements exceed this program's thresholds."
    )

state.report.append(overall_result)

content = [
    Header("Final Coaster Report"),
    Header(state.coaster_name, 2),
    overall_result,
    Table(
        [
            ["Total track elements", str(len(state.elements))],
            ["Elements within limits",
             str(len(state.elements) - elements_over_limit)],
            ["Elements over limits", str(elements_over_limit)]
        ],
        header=["Summary", "Count"]
```

```
    ),
    "These are idealized estimates, not an assessment of real ride safety.",
    HorizontalRule()
]

element_number = 1

for element in state.elements:
    content.append(
        Header(str(element_number) + ". " + element.name, 2)
    )
    content.append(
        "Entry speed: "
        + str(round(element.entry_speed, 2))
        + " m/s"
    )

    force_rows = [
        [
            "Vertical (bottom/crest/level)",
            str(round(element.vertical_g, 2)) + " g",
            str(MIN_VERTICAL_G) + " to "
            + str(MAX_VERTICAL_G) + " g"
        ],
        [
            "Lateral",
            str(round(element.lateral_g, 2)) + " g",
            "Magnitude up to " + str(MAX_LATERAL_G) + " g"
        ]
    ]
```

```
if "forward_g" in element.parameters:
    force_rows.append([
        "Forward/backward",
        str(round(element.parameters["forward_g"], 2)) + " g",
        "Magnitude up to " + str(MAX_FORWARD_G) + " g"
    ])

if "top_vertical_g" in element.parameters:
    force_rows.append([
        "Vertical at loop top",
        str(round(element.parameters["top_vertical_g"], 2))
        + " g",
        str(MIN_VERTICAL_G) + " to "
        + str(MAX_VERTICAL_G) + " g"
    ])
    content.append(
        "Loop top speed: "
        + str(round(element.parameters["top_speed"], 2))
        + " m/s"
    )

content.append(
    Table(
        force_rows,
        header=["Force", "Calculated value", "Program limit"]
    )
)

if within_limits(element):
    content.append("✓ Within program limits")
else:
```

```
            content.append("⚠ Exceeds program limits")


        content.append(HorizontalRule())
        element_number += 1


    content.append(
        Button("Start a New Coaster", "restart_action")
    )


    return Page(state, content)
```

This makes the report easier to scan because the **calculated values sit directly beside their limits**, instead of appearing among many bullet points.

Logan(11:29):

that looks really good. can you make all the other pages look like that

ChatGPT(11:37):

Yes—I'd give every main page the same structure: **heading, short instructions, summary table, divider, and inputs or buttons**.

Replace these five routes with the versions below. Keep your calculation routes and the improved report route.

```
"""Build a coaster and compare idealized rider forces with program limits.


Physics model:
* Ignore friction and air resistance.
* A drop converts gravitational potential energy into kinetic energy.
* A circular loop is evaluated at its bottom and top. It returns to its
  starting height, so its exit speed equals its entry speed.
* An airtime hill evaluates the crest only. Approach and descent return to
  the same height, so its exit speed also equals its entry speed.
* A banked turn is level. Rotate gravity and centripetal acceleration into
  the rider's vertical and lateral axes.
* Launch and brake acceleration are constant over their duration.
* These are educational estimates, not real ride safety assessments.


Python techniques used:
```

```
* list[TrackElement] stores dataclass instances.

* dict[str, float] stores named numeric parameters.

* math.isfinite rejects infinity and NaN.

* math.radians converts degrees for math.sin and math.cos.

* Dictionary.get supplies a default for an optional parameter.

* A default bank_angle argument lets different elements share one route.

* String.strip removes surrounding spaces.

* List.extend adds several report lines; append adds one item.

* in/not in check supported choices and optional dictionary keys.

* Chained comparisons check inclusive ranges.

* Index -1 accesses the final element after checking the list is not empty.

* Tests create local State objects without changing the live website.

"""


from drafter import *

from bakery import assert_equal

from dataclasses import dataclass

import math




set_website_title("Roller Coaster G-Force Simulator")

set_site_information(

    author="Logan Gagliardo",

    description="Create coaster elements and simulate forces on the rider.",

    sources="Idealized formulas from Newton's laws and conservation of energy.",

    planning="Copy of CS1 Final Project 1_ Design.docx",

    links=["https://github.com/logangag/coaster-gforce-simulator"]

)

hide_debug_information()

set_website_framed(False)
```

```
# Global constants are allowed by the project requirements.
GRAVITY = 9.81
BRAKE_DURATION = 5.0

MAX_VERTICAL_G = 4.5
MIN_VERTICAL_G = -1.0
MAX_LATERAL_G = 1.5
MAX_FORWARD_G = 1.5



@dataclass
class TrackElement:
    """Store one element and its calculated rider forces in units of g."""

    name: str
    element_type: str
    parameters: dict[str, float]
    entry_speed: float
    vertical_g: float
    lateral_g: float



@dataclass
class State:
    """Store the coaster, selected element, current speed, and report."""

    coaster_name: str
    elements: list[TrackElement]
    current_element_type: str
    current_speed: float
```

```
    report: list[str]



# HELPER FUNCTIONS


def initial_state() -> State:
    """Create an empty coaster with fresh lists."""
    return State("", [], "", 0.0, [])



def positive_number(value: float) -> bool:
    """Return whether a number is finite and greater than zero."""
    return math.isfinite(value) and value > 0



def invalid_selection(state: State) -> Page:
    """Display an unsupported-selection message with a recovery action."""
    return Page(state, [
        Header("Invalid Selection"),
        "Choose an available track element to continue.",
        HorizontalRule(),
        Table(
            [
                ["Vertical Loop", "Circular loop with bottom and top forces."],
                ["Airtime Hill", "Vertical force at a hill crest."],
                ["Banked Turn", "Vertical and lateral forces in a level turn."]
            ],
            header=["Available Element", "Description"]
        ),
        Button("Return to Builder", "element_selection_page")
    ])
```

```
def element_error(state: State, message: str, radius: float,
                  bank_angle: float) -> Page:
    """Display a consistent error screen while preserving submitted inputs."""
    content = [
        Header("Invalid Input"),
        "Check the settings below and try again.",
        HorizontalRule(),
        Table(
            [
                ["Selected element", state.current_element_type],
                ["Problem", message]
            ],
            header=["Detail", "Information"]
        ),
        Header("Update Your Settings", 2),
        "Curve radius (meters):",
        TextBox("radius", radius)
    ]

    if state.current_element_type == "Banked Turn":
        content.append("Bank angle (0 to 90 degrees):")
        content.append(TextBox("bank_angle", bank_angle))

    content.append(Button("Try Again", "add_element_action"))

    return Page(state, content)



def calculate_element(element_type: str, speed: float, radius: float,
```

```
                bank_angle: float) -> TrackElement:
    """Calculate forces for an already validated loop, hill, or level turn.

    Centripetal acceleration is speed squared divided by radius.
    Dividing by GRAVITY expresses it in units of g.

    A circular loop's top is 2 * radius above its bottom. Conservation
    of energy gives top_speed_squared = speed squared - 4 * g * radius.
    At the top, gravity points toward the center, so subtract 1 g when
    calculating the seat's force on the rider.
    """
    centripetal_g = speed ** 2 / (radius * GRAVITY)
    vertical_g = 1.0
    lateral_g = 0.0

    parameters = {
        "radius": radius,
        "bank_angle": bank_angle
    }

    if element_type == "Vertical Loop":
        vertical_g = 1 + centripetal_g

        top_speed_squared = speed ** 2 - 4 * GRAVITY * radius

        parameters["top_speed"] = math.sqrt(top_speed_squared)
        parameters["top_vertical_g"] = (
            top_speed_squared / (radius * GRAVITY) - 1
        )

    elif element_type == "Airtime Hill":
```

```
        vertical_g = 1 - centripetal_g

    elif element_type == "Banked Turn":
        angle = math.radians(bank_angle)

        vertical_g = (
            math.cos(angle) + centripetal_g * math.sin(angle)
        )
        lateral_g = (
            centripetal_g * math.cos(angle) - math.sin(angle)
        )

    return TrackElement(
        element_type,
        element_type,
        parameters,
        speed,
        vertical_g,
        lateral_g
    )



def within_limits(element: TrackElement) -> bool:
    """Check vertical, lateral, longitudinal, and optional loop-top forces."""
    if not MIN_VERTICAL_G <= element.vertical_g <= MAX_VERTICAL_G:
        return False

    if abs(element.lateral_g) > MAX_LATERAL_G:
        return False

    if abs(element.parameters.get("forward_g", 0.0)) > MAX_FORWARD_G:
```

```
        return False

    if "top_vertical_g" in element.parameters:
        top_g = element.parameters["top_vertical_g"]

        if not MIN_VERTICAL_G <= top_g <= MAX_VERTICAL_G:
            return False

    return True



def describe_element(element: TrackElement) -> list[str]:
    """Build report lines showing an element's forces and threshold result."""
    lines = [
        element.name,
        "Entry speed: " + str(round(element.entry_speed, 2)) + " m/s",
        "Vertical G-force (bottom/crest/level): "
        + str(round(element.vertical_g, 2)),
        "Lateral G-force: " + str(round(element.lateral_g, 2))
    ]

    if "forward_g" in element.parameters:
        lines.append(
            "Forward/backward G-force: "
            + str(round(element.parameters["forward_g"], 2))
        )

    if "top_vertical_g" in element.parameters:
        lines.append(
            "Loop top speed: "
            + str(round(element.parameters["top_speed"], 2))
```

```
            + " m/s"
        )
        lines.append(
            "Loop top vertical G-force: "
            + str(round(element.parameters["top_vertical_g"], 2))
        )

    if within_limits(element):
        lines.append("Element result: Within program limits")
    else:
        lines.append("Element result: EXCEEDS PROGRAM LIMIT")

    return lines


# HOME PAGE

@route
def index(state: State) -> Page:
    """Display the simulator overview and new-coaster inputs."""
    return Page(state, [
        Header("Roller Coaster G-Force Simulator"),
        "Design your coaster. Explore the forces. Review the results.",
        HorizontalRule(),

        Header("How It Works", 2),
        Table(
            [
                ["1. Start", "Name your coaster and choose a launch or drop."],
                ["2. Build", "Add loops, airtime hills, and banked turns."],
                ["3. Review", "Compare calculated forces with program limits."]
```

```
            ],
            header=["Step", "What You Do"]
        ),

        Header("Create Your Coaster", 2),
        "Coaster name:",
        TextBox("coaster_name", state.coaster_name),
        "Choose your first track element:",
        SelectBox("first_element", ["Launch", "90-degree Drop"]),
        Button("Start Building", "begin_coaster"),

        HorizontalRule(),
        Header("About the Simulation", 2),
        "This educational model ignores friction and air resistance.",
        "Loops and hills return to their entry height.",
        "Results are estimates, not a real ride safety assessment."
    ])



@route
def begin_coaster(state: State, coaster_name: str,
                  first_element: str) -> Page:
    """Reset the coaster and open the selected starting-element form."""
    if first_element != "Launch" and first_element != "90-degree Drop":
        return invalid_selection(state)

    state.coaster_name = coaster_name.strip()

    if state.coaster_name == "":
        state.coaster_name = "Untitled Coaster"
```

```
    state.current_element_type = first_element
    state.elements = []
    state.current_speed = 0.0
    state.report = []

    if first_element == "Launch":
        return launch_page(state)
    else:
        return drop_page(state)



# LAUNCH

@route
def launch_page(state: State) -> Page:
    """Display launch instructions, input guidance, and configuration fields."""
    return Page(state, [
        Header("Configure Your Launch"),
        "Give your coaster its starting speed with a powered launch.",
        HorizontalRule(),

        Header("Launch Settings", 2),
        Table(
            [
                ["Acceleration", "How quickly speed increases.", "m/s²"],
                ["Duration", "How long the launch accelerates.", "seconds"]
            ],
            header=["Setting", "Meaning", "Unit"]
        ),
        "Higher acceleration creates more forward force.",
        "A longer launch increases speed without changing its acceleration.",
```

```
        HorizontalRule(),
        Header("Configure Launch", 2),
        "Launch acceleration (m/s²):",
        TextBox("acceleration", 10.0),
        "Launch duration (seconds):",
        TextBox("duration", 3.0),
        "Both values must be greater than zero.",
        Button("Add Launch", "save_launch")
    ])



@route
def save_launch(state: State, acceleration: float,
                duration: float) -> Page:
    """Validate launch inputs, save the launch, and update the speed."""
    if not positive_number(acceleration) or not positive_number(duration):
        return Page(state, [
            Header("Invalid Input"),
            HorizontalRule(),
            "Acceleration and duration must be finite numbers greater than zero.",
            "Enter launch acceleration (m/s²):",
            TextBox("acceleration", acceleration),
            "Enter launch duration (seconds):",
            TextBox("duration", duration),
            Button("Try Again", "save_launch")
        ])

    entry_speed = state.current_speed
    forward_g = acceleration / GRAVITY
    new_speed = entry_speed + acceleration * duration
```

```
    launch = TrackElement(
        "Launch",
        "Launch",
        {
            "acceleration": acceleration,
            "duration": duration,
            "forward_g": forward_g
        },
        entry_speed,
        1.0,
        0.0
    )

    state.elements.append(launch)
    state.current_speed = new_speed

    return element_selection_page(state)



# DROP


@route
def drop_page(state: State) -> Page:
    """Display drop instructions, geometry guidance, and configuration fields."""
    return Page(state, [
        Header("Configure Your 90-Degree Drop"),
        "Turn height into speed, then curve out of the drop.",
        HorizontalRule(),

        Header("Drop Settings", 2),
```

```
        Table(
            [
                ["Drop height", "Vertical distance traveled downward.", "meters"],
                ["Curve radius", "Size of the curve at the bottom.", "meters"]
            ],
            header=["Setting", "Meaning", "Unit"]
        ),
        "A taller drop produces a higher bottom speed.",
        "A larger bottom radius reduces the calculated vertical force.",

        HorizontalRule(),
        Header("Configure Drop", 2),
        "Vertical drop height (meters):",
        TextBox("drop_height", 30.0),
        "Bottom curve radius (meters):",
        TextBox("curve_radius", 15.0),
        "Both values must be greater than zero.",
        Button("Add Drop", "save_drop")
    ])



@route
def save_drop(state: State, drop_height: float,
              curve_radius: float) -> Page:
    """Validate drop inputs and calculate bottom speed and rider force."""
    if not positive_number(drop_height) or not positive_number(curve_radius):
        return Page(state, [
            Header("Invalid Input"),
            HorizontalRule(),
            "Drop height and curve radius must be finite numbers greater than zero.",
            "Enter drop height (meters):",
```

```
        TextBox("drop_height", drop_height),
        "Enter curve radius (meters):",
        TextBox("curve_radius", curve_radius),
        Button("Try Again", "save_drop")
    ])

entry_speed = state.current_speed

# Conservation of energy: final speed squared = initial speed squared
# plus twice gravity times the vertical drop height.
new_speed = math.sqrt(
    entry_speed ** 2 + 2 * GRAVITY * drop_height
)

# At the bottom, the seat supports gravity and supplies the additional
# upward centripetal acceleration.
vertical_g = 1 + new_speed ** 2 / (curve_radius * GRAVITY)

drop = TrackElement(
    "90-Degree Drop",
    "Drop",
    {
        "drop_height": drop_height,
        "curve_radius": curve_radius
    },
    entry_speed,
    vertical_g,
    0.0
)

state.elements.append(drop)
```

```
    state.current_speed = new_speed


    return element_selection_page(state)



# ADDITIONAL ELEMENTS


@route
def element_selection_page(state: State) -> Page:
    """Display coaster progress and explain the available next elements."""
    content = [
        Header("Build Your Coaster"),
        "Choose the next experience for your riders.",
        HorizontalRule(),


        Header("Your Coaster", 2),
        Table(
            [
                ["Name", state.coaster_name],
                ["Current speed",
                 str(round(state.current_speed, 2)) + " m/s"],
                ["Elements added", str(len(state.elements))]
            ],
            header=["Detail", "Current Value"]
        )
    ]


    if state.elements:
        track_rows = []
        element_number = 1
```

```
    for element in state.elements:
        track_rows.append([
            str(element_number),
            element.name,
            str(round(element.entry_speed, 2)) + " m/s"
        ])
        element_number += 1

    content.append(Header("Track So Far", 2))
    content.append(
        Table(
            track_rows,
            header=["Order", "Element", "Entry Speed"]
        )
    )

content.extend([
    HorizontalRule(),
    Header("Choose Your Next Element", 2),
    Table(
        [
            ["Vertical Loop",
             "Compare forces at the bottom and top of a circular loop."],
            ["Airtime Hill",
             "Explore reduced vertical force at the hill's crest."],
            ["Banked Turn",
             "Explore how banking changes vertical and lateral forces."]
        ],
        header=["Element", "What It Simulates"]
    ),
    SelectBox(
```

```
            "element_type",
            ["Vertical Loop", "Airtime Hill", "Banked Turn"]
        ),
        Button("Configure Element", "choose_element"),

        HorizontalRule(),
        Header("Ready for Your Results?", 2),
        "Finishing adds a brake run and generates your coaster report.",
        Button("Finish Coaster", "finish_track")
    ])

    return Page(state, content)



@route
def choose_element(state: State, element_type: str) -> Page:
    """Save a supported element selection and display its input form."""
    if element_type not in ["Vertical Loop", "Airtime Hill", "Banked Turn"]:
        return invalid_selection(state)

    state.current_element_type = element_type

    return element_config_page(state)



@route
def element_config_page(state: State) -> Page:
    """Display an explanation and configuration inputs for the chosen element."""
    content = [
        Header("Configure " + state.current_element_type),
        "Adjust the geometry and explore its effect on rider forces.",
```

```
    HorizontalRule(),


    Header("Current Conditions", 2),
    Table(
        [
            ["Coaster", state.coaster_name],
            ["Selected element", state.current_element_type],
            ["Entry speed",
             str(round(state.current_speed, 2)) + " m/s"]
        ],
        header=["Detail", "Value"]
    ),
    HorizontalRule(),
    Header("Element Settings", 2)
]


if state.current_element_type == "Banked Turn":
    content.extend([
        Table(
            [
                ["Radius", "A larger radius makes the turn gentler."],
                ["Bank angle",
                 "Tilts the rider and changes the force distribution."]
            ],
            header=["Setting", "Effect"]
        ),
        "Curve radius (meters):",
        TextBox("radius", 20.0),
        "Bank angle (0 to 90 degrees):",
        TextBox("bank_angle", 30.0)
    ])
```

```
    elif state.current_element_type == "Vertical Loop":
        content.extend([
            "The report evaluates forces at both the loop bottom and top.",
            "A larger loop requires more entry speed to complete.",
            "The program checks whether your coaster has enough speed.",
            "Loop radius (meters):",
            TextBox("radius", 20.0)
        ])

    else:
        content.extend([
            "The calculated vertical force is evaluated at the hill's crest.",
            "A smaller radius produces a stronger reduction in vertical force.",
            "Hill crest radius (meters):",
            TextBox("radius", 20.0)
        ])

    content.extend([
        "Radius must be greater than zero.",
        Button("Add Element", "add_element_action")
    ])

    return Page(state, content)



@route
def add_element_action(state: State, radius: float,
                       bank_angle: float = 0.0) -> Page:
    """Validate geometry and loop speed, then append the calculated element."""
    if not positive_number(radius):
```

```
    return element_error(
        state,
        "Radius must be a finite number greater than zero.",
        radius,
        bank_angle
    )

if state.current_element_type not in [
    "Vertical Loop", "Airtime Hill", "Banked Turn"
]:
    return invalid_selection(state)

if state.current_element_type == "Banked Turn":
    if not math.isfinite(bank_angle) or bank_angle < 0 or bank_angle > 90:
        return element_error(
            state,
            "Bank angle must be a finite number from 0 to 90 degrees.",
            radius,
            bank_angle
        )

if state.current_element_type == "Vertical Loop":
    # An unpowered circular loop needs v squared >= 5*g*r to keep
    # nonnegative seat force at the top under this idealized model.
    if state.current_speed ** 2 < 5 * GRAVITY * radius:
        return element_error(
            state,
            "Too slow to complete this circular loop. Use a smaller radius.",
            radius,
            bank_angle
        )
```

```
    element = calculate_element(
        state.current_element_type,
        state.current_speed,
        radius,
        bank_angle
    )

    state.elements.append(element)

    return element_selection_page(state)



# FINISH AND REPORT


@route
def finish_track(state: State) -> Page:
    """Add one final brake run, stop the coaster, and show its report."""
    if not state.elements:
        return index(state)

    # Repeated submissions should not add duplicate brake runs.
    if state.elements[-1].element_type == "Brake Run":
        return report_page(state)

    entry_speed = state.current_speed
    deceleration = entry_speed / BRAKE_DURATION
    forward_g = -deceleration / GRAVITY

    brake_run = TrackElement(
        "Required Brake Run",
```

```
        "Brake Run",
        {
            "deceleration": deceleration,
            "duration": BRAKE_DURATION,
            "forward_g": forward_g
        },
        entry_speed,
        1.0,
        0.0
    )

    state.elements.append(brake_run)
    state.current_speed = 0.0

    return report_page(state)


@route
def report_page(state: State) -> Page:
    """Display a summary and a force table for each track element.

    Keep the text report in State for testing and future export features.
    Table uses rows of cell values and a separate header row.
    HorizontalRule visually separates the element sections.
    """
    state.report = [
        "Coaster: " + state.coaster_name,
        "Total track elements: " + str(len(state.elements))
    ]

    overall_safe = True
```

```
elements_over_limit = 0


for element in state.elements:
    state.report.extend(describe_element(element))


    if not within_limits(element):
        overall_safe = False
        elements_over_limit += 1


if overall_safe:
    overall_result = (
        "Overall Result: Within this program's thresholds."
    )
else:
    overall_result = (
        "Overall Result: Some elements exceed this program's thresholds."
    )


state.report.append(overall_result)


content = [
    Header("Final Coaster Report"),
    Header(state.coaster_name, 2),
    overall_result,
    Table(
        [
            ["Total track elements", str(len(state.elements))],
            ["Elements within limits",
             str(len(state.elements) - elements_over_limit)],
            ["Elements over limits", str(elements_over_limit)]
        ],
```

```
        header=["Summary", "Count"]
    ),
    "These are idealized estimates, not an assessment of real ride safety.",
    HorizontalRule()
]

element_number = 1

for element in state.elements:
    content.append(
        Header(str(element_number) + ". " + element.name, 2)
    )
    content.append(
        "Entry speed: "
        + str(round(element.entry_speed, 2))
        + " m/s"
    )

    force_rows = [
        [
            "Vertical (bottom/crest/level)",
            str(round(element.vertical_g, 2)) + " g",
            str(MIN_VERTICAL_G) + " to "
            + str(MAX_VERTICAL_G) + " g"
        ],
        [
            "Lateral",
            str(round(element.lateral_g, 2)) + " g",
            "Magnitude up to " + str(MAX_LATERAL_G) + " g"
        ]
    ]
```

```
if "forward_g" in element.parameters:
    force_rows.append([
        "Forward/backward",
        str(round(element.parameters["forward_g"], 2)) + " g",
        "Magnitude up to " + str(MAX_FORWARD_G) + " g"
    ])

if "top_vertical_g" in element.parameters:
    force_rows.append([
        "Vertical at loop top",
        str(round(element.parameters["top_vertical_g"], 2))
        + " g",
        str(MIN_VERTICAL_G) + " to "
        + str(MAX_VERTICAL_G) + " g"
    ])
    content.append(
        "Loop top speed: "
        + str(round(element.parameters["top_speed"], 2))
        + " m/s"
    )

content.append(
    Table(
        force_rows,
        header=["Force", "Calculated value", "Program limit"]
    )
)

if within_limits(element):
    content.append("✓ Within program limits")
```

```
            else:
                content.append("⚠ Exceeds program limits")

            content.append(HorizontalRule())
            element_number += 1

    content.append(
        Button("Start a New Coaster", "restart_action")
    )

    return Page(state, content)


@route
def restart_action(state: State) -> Page:
    """Clear every state field and return to the home page."""
    state.coaster_name = ""
    state.elements = []
    state.current_element_type = ""
    state.current_speed = 0.0
    state.report = []

    return index(state)


# UNIT TESTS
# Every route is deterministic and can be unit tested.
# Tests run before the server starts and use separate local State objects.
# Round calculated floats to avoid insignificant representation differences.
# float("inf") and float("nan") create special values for validation tests.
```

```
def test_index() -> None:
    """Check the redesigned home page's input and start action."""
    state = initial_state()
    page = index(state)

    assert_equal(page.state, state)
    assert_equal(
        page.content[0],
        Header("Roller Coaster G-Force Simulator")
    )
    assert_equal(TextBox("coaster_name", "") in page.content, True)
    assert_equal(
        Button("Start Building", "begin_coaster") in page.content,
        True
    )



def test_begin_coaster() -> None:
    """Check starting choices, reset behavior, and blank-name handling."""
    state = State("Old", [], "Banked Turn", 50.0, ["Old report"])
    state.elements.append(
        TrackElement("Old", "Old", {}, 0.0, 1.0, 0.0)
    )

    page = begin_coaster(state, "  Comet  ", "Launch")

    assert_equal(state, State("Comet", [], "Launch", 0.0, []))
    assert_equal(page.content[0], Header("Configure Your Launch"))

    page = begin_coaster(state, "    ", "90-degree Drop")
```

```
    assert_equal(state.coaster_name, "Untitled Coaster")
    assert_equal(
        page.content[0],
        Header("Configure Your 90-Degree Drop")
    )

    page = begin_coaster(state, "Ignored", "Unsupported")

    assert_equal(page.content[0], Header("Invalid Selection"))
    assert_equal(state.coaster_name, "Untitled Coaster")


def test_launch_page() -> None:
    """Check that the redesigned launch form retains its required inputs."""
    page = launch_page(initial_state())

    assert_equal(page.content[0], Header("Configure Your Launch"))
    assert_equal(TextBox("acceleration", 10.0) in page.content, True)
    assert_equal(TextBox("duration", 3.0) in page.content, True)
    assert_equal(
        Button("Add Launch", "save_launch") in page.content,
        True
    )


def test_save_launch() -> None:
    """Check launch speed, stored parameters, and invalid input rejection."""
    state = initial_state()
    page = save_launch(state, 10.0, 3.0)

    assert_equal(state.current_speed, 30.0)
```

```
assert_equal(state.elements, [TrackElement(
    "Launch",
    "Launch",
    {
        "acceleration": 10.0,
        "duration": 3.0,
        "forward_g": 10.0 / GRAVITY
    },
    0.0,
    1.0,
    0.0
)])
assert_equal(page.content[0], Header("Build Your Coaster"))

for acceleration in [-1.0, 0.0, float("inf"), float("nan")]:
    page = save_launch(state, acceleration, 3.0)

    assert_equal(page.content[0], Header("Invalid Input"))
    assert_equal(len(state.elements), 1)
    assert_equal(state.current_speed, 30.0)

page = save_launch(state, 10.0, 0.0)

assert_equal(page.content[0], Header("Invalid Input"))
assert_equal(len(state.elements), 1)

save_launch(state, 2.0, 5.0)

assert_equal(state.elements[-1].entry_speed, 30.0)
assert_equal(state.current_speed, 40.0)
```

```
def test_drop_page() -> None:
    """Check that the redesigned drop form retains its required inputs."""
    page = drop_page(initial_state())

    assert_equal(
        page.content[0],
        Header("Configure Your 90-Degree Drop")
    )
    assert_equal(TextBox("drop_height", 30.0) in page.content, True)
    assert_equal(TextBox("curve_radius", 15.0) in page.content, True)
    assert_equal(Button("Add Drop", "save_drop") in page.content, True)



def test_save_drop() -> None:
    """Check energy-based speed, bottom force, and invalid geometry."""
    state = initial_state()
    page = save_drop(state, 30.0, 15.0)

    assert_equal(round(state.current_speed, 4), 24.2611)
    assert_equal(state.elements[0].vertical_g, 5.0)
    assert_equal(state.elements[0].entry_speed, 0.0)
    assert_equal(page.content[0], Header("Build Your Coaster"))

    for height in [-1.0, 0.0, float("inf"), float("nan")]:
        page = save_drop(state, height, 15.0)

        assert_equal(page.content[0], Header("Invalid Input"))
        assert_equal(len(state.elements), 1)

    for radius in [-1.0, 0.0, float("inf"), float("nan")]:
```

        page = save_drop(state, 30.0, radius)

        assert_equal(page.content[0], Header("Invalid Input"))
        assert_equal(len(state.elements), 1)

    state.current_speed = 10.0
    save_drop(state, 10.0, 20.0)

    assert_equal(round(state.current_speed, 4), 17.2105)

def test_element_selection_page() -> None:
    """Check that the builder summary displays the current coaster values."""
    state = State("Comet", [], "Launch", 30.0, [])
    page = element_selection_page(state)

    assert_equal(page.content[0], Header("Build Your Coaster"))
    assert_equal(
        Table(
            [
                ["Name", "Comet"],
                ["Current speed", "30.0 m/s"],
                ["Elements added", "0"]
            ],
            header=["Detail", "Current Value"]
) in page.content,
        True
)
    assert_equal(
        Button("Finish Coaster", "finish_track") in page.content,
        True

```
    )



def test_choose_element() -> None:
    """Check that valid selections change state and invalid ones do not."""
    state = initial_state()

    for kind in ["Vertical Loop", "Airtime Hill", "Banked Turn"]:
        page = choose_element(state, kind)

        assert_equal(state.current_element_type, kind)
        assert_equal(page.content[0], Header("Configure " + kind))

    page = choose_element(state, "Unsupported")

    assert_equal(state.current_element_type, "Banked Turn")
    assert_equal(page.content[0], Header("Invalid Selection"))



def test_element_config_page() -> None:
    """Check angle-field visibility and radius inputs for each element."""
    state = initial_state()

    for kind in ["Banked Turn", "Vertical Loop", "Airtime Hill"]:
        state.current_element_type = kind
        page = element_config_page(state)

        assert_equal(page.content[0], Header("Configure " + kind))
        assert_equal(TextBox("radius", 20.0) in page.content, True)
        assert_equal(
            TextBox("bank_angle", 30.0) in page.content,
```

```
            kind == "Banked Turn"
        )


def test_add_element_action() -> None:
    """Check force models, loop feasibility, and preserved error inputs."""
    state = State("Comet", [], "Banked Turn", 30.0, [])
    page = add_element_action(state, 20.0, 0.0)

    assert_equal(page.content[0], Header("Build Your Coaster"))
    assert_equal(round(state.elements[-1].lateral_g, 4), 4.5872)
    assert_equal(state.elements[-1].vertical_g, 1.0)

    add_element_action(state, 20.0, 30.0)

    assert_equal(round(state.elements[-1].lateral_g, 4), 3.4726)
    assert_equal(round(state.elements[-1].vertical_g, 4), 3.1596)

    page = add_element_action(state, 0.0, 45.0)

    assert_equal(TextBox("bank_angle", 45.0) in page.content, True)
    assert_equal(len(state.elements), 2)

    for angle in [-1.0, 91.0, float("inf"), float("nan")]:
        page = add_element_action(state, 20.0, angle)

        assert_equal(page.content[0], Header("Invalid Input"))
        assert_equal(len(state.elements), 2)

    for radius in [-1.0, float("inf"), float("nan")]:
        page = add_element_action(state, radius, 45.0)
```

```
    assert_equal(page.content[0], Header("Invalid Input"))
    assert_equal(len(state.elements), 2)

state.current_element_type = "Airtime Hill"
add_element_action(state, 20.0)

assert_equal(round(state.elements[-1].vertical_g, 4), -3.5872)

state.current_element_type = "Vertical Loop"
page = add_element_action(state, 20.0)

assert_equal(page.content[0], Header("Invalid Input"))
assert_equal(len(state.elements), 3)

add_element_action(state, 10.0)

assert_equal(round(state.elements[-1].vertical_g, 4), 10.1743)
assert_equal(
    round(state.elements[-1].parameters["top_vertical_g"], 4),
    4.1743
)
assert_equal(
    round(state.elements[-1].parameters["top_speed"], 4),
    22.5300
)
assert_equal(state.current_speed, 30.0)

state.current_element_type = "Unsupported"
page = add_element_action(state, 10.0)
```

```
    assert_equal(page.content[0], Header("Invalid Selection"))
    assert_equal(len(state.elements), 4)



def test_finish_track() -> None:
    """Check braking, stopping, and protection from duplicate brake runs."""
    state = initial_state()
    page = finish_track(state)

    assert_equal(
        page.content[0],
        Header("Roller Coaster G-Force Simulator")
    )
    assert_equal(state.elements, [])

    save_launch(state, 10.0, 3.0)
    page = finish_track(state)

    assert_equal(page.content[0], Header("Final Coaster Report"))
    assert_equal(len(state.elements), 2)
    assert_equal(state.current_speed, 0.0)
    assert_equal(state.elements[-1].element_type, "Brake Run")
    assert_equal(state.elements[-1].entry_speed, 30.0)
    assert_equal(state.elements[-1].parameters["deceleration"], 6.0)
    assert_equal(
        round(state.elements[-1].parameters["forward_g"], 4),
        -0.6116
    )

    finish_track(state)
```

```
    assert_equal(len(state.elements), 2)



def test_report_page() -> None:
    """Check passing and failing reports and rebuilding without duplicates."""
    state = State(
        "Comet",
        [TrackElement("Gentle", "Banked Turn", {}, 10.0, 1.0, 0.5)],
        "Banked Turn",
        10.0,
        []
    )
    page = report_page(state)

    assert_equal(page.content[0], Header("Final Coaster Report"))
    assert_equal(state.report[0], "Coaster: Comet")
    assert_equal(
        state.report[-1],
        "Overall Result: Within this program's thresholds."
    )
    assert_equal("Gentle" in state.report, True)
    assert_equal(len(state.report), 8)

    report_page(state)

    assert_equal(len(state.report), 8)

    state.elements.append(
        TrackElement("Sharp", "Airtime Hill", {}, 30.0, -2.0, 0.0)
    )
    report_page(state)
```

```
    assert_equal("Sharp" in state.report, True)
    assert_equal(
        "Element result: EXCEEDS PROGRAM LIMIT" in state.report,
        True
    )
    assert_equal(
        state.report[-1],
        "Overall Result: Some elements exceed this program's thresholds."
    )



def test_restart_action() -> None:
    """Check that restart clears every field and returns the home page."""
    state = State(
        "Comet",
        [TrackElement("Old", "Launch", {}, 0.0, 1.0, 0.0)],
        "Launch",
        30.0,
        ["Old report"]
    )
    page = restart_action(state)

    assert_equal(state, initial_state())
    assert_equal(
        page.content[0],
        Header("Roller Coaster G-Force Simulator")
    )



def test_helpers() -> None:
```

```
"""Check inclusive limits, loop-top limits, and balanced banking."""
assert_equal(positive_number(0.0), False)
assert_equal(positive_number(2.0), True)
assert_equal(positive_number(float("nan")), False)
assert_equal(positive_number(float("inf")), False)

element = TrackElement(
    "Test", "Test", {"forward_g": 1.5}, 10.0, 4.5, 1.5
)
assert_equal(within_limits(element), True)

element.vertical_g = -1.0
assert_equal(within_limits(element), True)

element.vertical_g = -1.01
assert_equal(within_limits(element), False)

element.vertical_g = 4.51
assert_equal(within_limits(element), False)

element.vertical_g = 1.0
element.lateral_g = -1.51
assert_equal(within_limits(element), False)

element.lateral_g = -1.5
element.parameters["forward_g"] = -1.5
assert_equal(within_limits(element), True)

element.parameters["forward_g"] = -1.51
assert_equal(within_limits(element), False)
```

```
    element.parameters = {
        "top_vertical_g": 4.51,
        "top_speed": 20.0
    }
    assert_equal(within_limits(element), False)

    element.parameters["top_vertical_g"] = -1.01
    assert_equal(within_limits(element), False)

    # Centripetal acceleration is exactly 1 g here. Banking 45 degrees
    # removes lateral force and produces sqrt(2) g in the vertical axis.
    balanced = calculate_element(
        "Banked Turn",
        math.sqrt(20 * GRAVITY),
        20.0,
        45.0
    )

    assert_equal(round(balanced.lateral_g, 4), 0.0)
    assert_equal(round(balanced.vertical_g, 4), 1.4142)
    assert_equal(
        "Element result: EXCEEDS PROGRAM LIMIT"
        in describe_element(element),
        True
    )



def run_tests() -> None:
    """Run Bakery tests for all thirteen routes and the helper functions."""
    test_index()
    test_begin_coaster()
```

```
    test_launch_page()

    test_save_launch()

    test_drop_page()

    test_save_drop()

    test_element_selection_page()

    test_choose_element()

    test_element_config_page()

    test_add_element_action()

    test_finish_track()

    test_report_page()

    test_restart_action()

    test_helpers()



run_tests()

start_server(initial_state())
```